In [1]:
```
import numpy as np
arr = np.array([10, 20, 30, 40, 50])
arr2 = np.array([1, 2, 3, 4, 5])
print("Array:", arr)
print("Second Array:", arr2)
```

```
Array: [10 20 30 40 50]
Second Array: [1 2 3 4 5]
```

In [2]:
```
#Inspect
print("Dimensions:", arr.ndim)
print("Shape:", arr.shape)
print("Size:", arr.size)
```

```
Dimensions: 1
Shape: (5,)
Size: 5
```

In [3]:
```
print("Minimum:", np.min(arr))
print("Maximum:", np.max(arr))
print("Sum:", np.sum(arr))
print("Mean:", np.mean(arr))
```

```
Minimum: 10
Maximum: 50
Sum: 150
Mean: 30.0
```

In [7]:
```
#Manipulation

print("Array + 5:", arr + 5)
print("Array * 2:", arr * 2)
```

```
Array + 5: [15 25 35 45 55]
Array * 2: [ 20  40  60  80 100]
```

In [5]:
```
#Multiply
print("Array Multiplication:", arr * arr2)
```

```
Array Multiplication: [ 10  40  90 160 250]
```

In [6]:
```
# Filter
print("Elements greater than 25:", arr[arr > 25])
print("Even numbers:", arr[arr % 2 == 0])
```

```
Elements greater than 25: [30 40 50]
Even numbers: [10 20 30 40 50]
```

In [8]:
```
import numpy as np
import pandas as pd
arr2 = np.array([
    [100, 200, 150],
    [120, 250, 180],
    [90,  220, 160],
    [140, 300, 200],
    [110, 240, 170]
])
```

In [9]:
```
print("Array:", arr2)
print("Dimensions:", arr2.ndim)
print("Shape:", arr2.shape)
print("Size:", arr2.size)
```

```
Array:
 [[100 200 150]
 [120 250 180]
 [ 90 220 160]
 [140 300 200]
 [110 240 170]]
Dimensions: 2
Shape: (5, 3)
Size: 15
```

In [10]:
```
print("Sum of each row:", arr2.sum(axis=1))
print("Sum of each column:", arr2.sum(axis=0))
```

```
Sum of each row: [450 550 470 640 520]
Sum of each column: [ 560 1210  860]
```

In [11]:
```
#Statistics
print("Total:", arr2.sum())
print("Average:", arr2.mean())
print("Minimum:", arr2.min())
print("Maximum:", arr2.max())
```

```
Total: 2630
Average: 175.33333333333334
Minimum: 90
Maximum: 300
```

In [17]:
```
#pandas
import numpy as np
import pandas as pd
url = "https://raw.githubusercontent.com/plotly/datasets/master/supermarket
df = pd.read_csv(url)
```

In [18]:
```
print(df.head())
```

```
    Invoice ID Branch       City Customer type  Gender  \
0  750-67-8428      A     Yangon        Member  Female
1  226-31-3081      C  Naypyitaw        Normal  Female
2  631-41-3108      A     Yangon        Normal    Male
3  123-19-1176      A     Yangon        Member    Male
4  373-73-7910      A     Yangon        Normal    Male

             Product line  Unit price  Quantity   Tax 5%     Total       D
ate  \
0       Health and beauty       74.69         7  26.1415  548.9715   1/5/2
019
1  Electronic accessories       15.28         5   3.8200   80.2200   3/8/2
019
2      Home and lifestyle       46.33         7  16.2155  340.5255   3/3/2
019
3       Health and beauty       58.22         8  23.2880  489.0480  1/27/2
019
4       Sports and travel       86.31         7  30.2085  634.3785   2/8/2
019

    Time      Payment  Cost of goods sold  Gross margin percentage  \
0  13:08      Ewallet              522.83                 4.761905
1  10:29         Cash               76.40                 4.761905
2  13:23  Credit card              324.31                 4.761905
3  20:33      Ewallet              465.76                 4.761905
4  10:37      Ewallet              604.17                 4.761905

   Gross income  Customer stratification rating
0       26.1415                            9.1
1        3.8200                            9.6
2       16.2155                            7.4
3       23.2880                            8.4
4       30.2085                            5.3
```

In [23]:
```
df.tail()
```

Out[23]:

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	
995	233-67-5758	C	Naypyitaw	Normal	Male	Health and beauty	40.35	1	2.0175	
996	303-96-2227	B	Mandalay	Normal	Female	Home and lifestyle	97.38	10	48.6900	10
997	727-02-1313	A	Yangon	Member	Male	Food and beverages	31.84	1	1.5920	
998	347-56-2442	A	Yangon	Normal	Male	Home and lifestyle	65.82	1	3.2910	
999	849-09-3807	A	Yangon	Member	Female	Fashion accessories	88.34	7	30.9190	6

In [19]:
```
print(df.shape)
print(df.columns)
```

```
(1000, 17)
Index(['Invoice ID', 'Branch', 'City', 'Customer type', 'Gender',
       'Product line', 'Unit price', 'Quantity', 'Tax 5%', 'Total', 'Dat
e',
       'Time', 'Payment', 'Cost of goods sold', 'Gross margin percentage',
       'Gross income', 'Customer stratification rating'],
      dtype='object')
```

In [21]:
```
arr3 = df[["Unit price", "Quantity", "Total"]].to_numpy()
print("2D Array:")
print(arr3)
```

```
2D Array:
[[ 74.69     7.     548.9715]
 [ 15.28     5.      80.22  ]
 [ 46.33     7.     340.5255]
 ...
 [ 31.84     1.      33.432 ]
 [ 65.82     1.      69.111 ]
 [ 88.34     7.     649.299 ]]
```

In [22]:
```
print("Dimensions:", arr3.ndim)
print("Shape:", arr3.shape)
print("Size:", arr3.size)
```

```
Dimensions: 2
Shape: (1000, 3)
Size: 3000
```

In [25]:
```
print("SHAPE:")
print(df.shape)
```

```
SHAPE:
(1000, 17)
```

In [26]:
```
print("COLUMNS:")
print(df.columns)
```

```
COLUMNS:
Index(['Invoice ID', 'Branch', 'City', 'Customer type', 'Gender',
       'Product line', 'Unit price', 'Quantity', 'Tax 5%', 'Total', 'Dat
e',
       'Time', 'Payment', 'Cost of goods sold', 'Gross margin percentage',
       'Gross income', 'Customer stratification rating'],
      dtype='object')
```

In [27]:
```
print("INFO:")
df.info()
```

```
INFO:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 17 columns):
 #   Column                          Non-Null Count  Dtype  
---  ------                          --------------  -----  
 0   Invoice ID                      1000 non-null   object 
 1   Branch                          1000 non-null   object 
 2   City                            1000 non-null   object 
 3   Customer type                   1000 non-null   object 
 4   Gender                          1000 non-null   object 
 5   Product line                    1000 non-null   object 
 6   Unit price                      1000 non-null   float64
 7   Quantity                        1000 non-null   int64  
 8   Tax 5%                          1000 non-null   float64
 9   Total                           1000 non-null   float64
 10  Date                            1000 non-null   object 
 11  Time                            1000 non-null   object 
 12  Payment                         1000 non-null   object 
 13  Cost of goods sold              1000 non-null   float64
 14  Gross margin percentage         1000 non-null   float64
 15  Gross income                    1000 non-null   float64
 16  Customer stratification rating  1000 non-null   float64
dtypes: float64(7), int64(1), object(9)
memory usage: 132.9+ KB
```

In [28]:
```
print("DESCRIBE:")
print(df.describe())
```

```
DESCRIBE:
       Unit price     Quantity       Tax 5%        Total  Cost of goods s
old  \
count  1000.000000  1000.000000  1000.000000  1000.000000          1000.00
000
mean     55.672130     5.510000    15.379369   322.966749           307.58
738
std      26.494628     2.923431    11.708825   245.885335           234.17
651
min      10.080000     1.000000     0.508500    10.678500            10.17
000
25%      32.875000     3.000000     5.924875   124.422375           118.49
750
50%      55.230000     5.000000    12.088000   253.848000           241.76
000
75%      77.935000     8.000000    22.445250   471.350250           448.90
500
max      99.960000    10.000000    49.650000  1042.650000           993.00
000

       Gross margin percentage  Gross income  Customer stratification rati
ng
count             1.000000e+03   1000.000000                      1000.000
00
mean              4.761905e+00     15.379369                         6.972
70
std               6.131498e-14     11.708825                         1.718
58
min               4.761905e+00      0.508500                         4.000
00
25%               4.761905e+00      5.924875                         5.500
00
50%               4.761905e+00     12.088000                         7.000
00
75%               4.761905e+00     22.445250                         8.500
00
max               4.761905e+00     49.650000                        10.000
00
```

In [29]:
```
print("DESCRIBE ALL:")
print(df.describe(include="all"))
```

```
DESCRIBE ALL:
        Invoice ID Branch    City Customer type  Gender         Product l
ine  \
count         1000   1000    1000          1000    1000                1
000
unique        1000      3       3             2       2
6
top     750-67-8428      A  Yangon        Member  Female  Fashion accessor
ies
freq             1    340     340           501     501
178
mean           NaN    NaN     NaN           NaN     NaN
NaN
std            NaN    NaN     NaN           NaN     NaN
NaN
min            NaN    NaN     NaN           NaN     NaN
NaN
25%            NaN    NaN     NaN           NaN     NaN
NaN
50%            NaN    NaN     NaN           NaN     NaN
NaN
75%            NaN    NaN     NaN           NaN     NaN
NaN
max            NaN    NaN     NaN           NaN     NaN
NaN

         Unit price     Quantity       Tax 5%        Total      Date   Tim
e  \
count   1000.000000  1000.000000  1000.000000  1000.000000      1000   100
0
unique          NaN          NaN          NaN          NaN        89    50
6
top             NaN          NaN          NaN          NaN  2/7/2019  19:4
8
freq            NaN          NaN          NaN          NaN        20
7
mean      55.672130     5.510000    15.379369   322.966749       NaN    Na
N
std       26.494628     2.923431    11.708825   245.885335       NaN    Na
N
min       10.080000     1.000000     0.508500    10.678500       NaN    Na
N
25%       32.875000     3.000000     5.924875   124.422375       NaN    Na
N
50%       55.230000     5.000000    12.088000   253.848000       NaN    Na
N
75%       77.935000     8.000000    22.445250   471.350250       NaN    Na
N
max       99.960000    10.000000    49.650000  1042.650000       NaN    Na
N

        Payment  Cost of goods sold  Gross margin percentage  Gross income
\
count      1000          1000.00000             1.000000e+03   1000.000000
unique        3                 NaN                      NaN           NaN
top     Ewallet                 NaN                      NaN           NaN
freq        345                 NaN                      NaN           NaN
mean        NaN           307.58738             4.761905e+00     15.379369
std         NaN           234.17651             6.131498e-14     11.708825
min         NaN            10.17000             4.761905e+00      0.508500
```

```
25%          NaN          118.49750             4.761905e+00      5.924875
50%          NaN          241.76000             4.761905e+00     12.088000
75%          NaN          448.90500             4.761905e+00     22.445250
max          NaN          993.00000             4.761905e+00     49.650000

        Customer stratification rating
count                      1000.00000
unique                            NaN
top                               NaN
freq                              NaN
mean                          6.97270
std                           1.71858
min                           4.00000
25%                           5.50000
50%                           7.00000
75%                           8.50000
max                          10.00000
```

In [30]:
```
print("MISSING VALUES:")
print(df.isnull().sum())
```

```
MISSING VALUES:
Invoice ID                      0
Branch                          0
City                            0
Customer type                   0
Gender                          0
Product line                    0
Unit price                      0
Quantity                        0
Tax 5%                          0
Total                           0
Date                            0
Time                            0
Payment                         0
Cost of goods sold              0
Gross margin percentage         0
Gross income                    0
Customer stratification rating  0
dtype: int64
```

In [31]:
```
print("DUPLICATE ROWS:")
print(df.duplicated().sum())
```

```
DUPLICATE ROWS:
0
```

In [32]:
```
df["Calculated_Sales"] = df["Unit price"] * df["Quantity"]
df
```

Out[32]:

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	
0	750-67-8428	A	Yangon	Member	Female	Health and beauty	74.69	7	26.1415	5
1	226-31-3081	C	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.8200	
2	631-41-3108	A	Yangon	Normal	Male	Home and lifestyle	46.33	7	16.2155	3
3	123-19-1176	A	Yangon	Member	Male	Health and beauty	58.22	8	23.2880	4
4	373-73-7910	A	Yangon	Normal	Male	Sports and travel	86.31	7	30.2085	6
...	...	...	...	...	...	...	...	...	...	
995	233-67-5758	C	Naypyitaw	Normal	Male	Health and beauty	40.35	1	2.0175	
996	303-96-2227	B	Mandalay	Normal	Female	Home and lifestyle	97.38	10	48.6900	10
997	727-02-1313	A	Yangon	Member	Male	Food and beverages	31.84	1	1.5920	
998	347-56-2442	A	Yangon	Normal	Male	Home and lifestyle	65.82	1	3.2910	
999	849-09-3807	A	Yangon	Member	Female	Fashion accessories	88.34	7	30.9190	6

1000 rows × 18 columns



In [33]:
```
df["Discount"] = df["Calculated_Sales"] * 0.10
df
```

Out[33]:

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	
0	750-67-8428	A	Yangon	Member	Female	Health and beauty	74.69	7	26.1415	5
1	226-31-3081	C	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.8200	
2	631-41-3108	A	Yangon	Normal	Male	Home and lifestyle	46.33	7	16.2155	3
3	123-19-1176	A	Yangon	Member	Male	Health and beauty	58.22	8	23.2880	4
4	373-73-7910	A	Yangon	Normal	Male	Sports and travel	86.31	7	30.2085	6
...	...	...	...	...	...	...	...	...	...	
995	233-67-5758	C	Naypyitaw	Normal	Male	Health and beauty	40.35	1	2.0175	
996	303-96-2227	B	Mandalay	Normal	Female	Home and lifestyle	97.38	10	48.6900	10
997	727-02-1313	A	Yangon	Member	Male	Food and beverages	31.84	1	1.5920	
998	347-56-2442	A	Yangon	Normal	Male	Home and lifestyle	65.82	1	3.2910	
999	849-09-3807	A	Yangon	Member	Female	Fashion accessories	88.34	7	30.9190	6

1000 rows × 19 columns

In [34]:
```
arr4 = df[["Unit price", "Quantity", "Total"]].to_numpy()
arr4
```

```
array([[ 74.69,   7.  , 548.97],
       [ 15.28,   5.  , 80.22],
       [ 46.33,   7.  , 340.52],
       [ 58.22,   8.  , 489.048],
       [ 86.31,   7.  , 634.6785],
       ...,
       [ 40.35,   1.  , 42.3675],
       [ 97.38,  10.  , 1022.49],
       [ 31.84,   1.  , 33.432],
       [ 65.82,   1.  , 69.111],
       [ 88.34,   7.  , 649.299]])
```

In [35]:
```
# Sales greater than 500
high_sales = df[df["Total"] > 500]
print(high_sales)

# Quantity greater than 5
high_quantity = df[df["Quantity"] > 5]
print(high_quantity)

#and
filtered = df[(df["Total"] > 500) &(df["Quantity"] >= 5)]
print(filtered)

#or
filtered_or = df[(df["Total"] > 500) |(df["Quantity"] > 8)]
print(filtered_or)
```

```
      Invoice ID Branch       City Customer type  Gender  \
0    750-67-8428      A     Yangon        Member  Female
4    373-73-7910      A     Yangon        Normal    Male
5    699-14-3026      C  Naypyitaw        Normal    Male
7    315-22-5665      C  Naypyitaw        Normal  Female
14   829-34-3910      A     Yangon        Normal  Female
..           ...    ...        ...           ...     ...
988  267-62-7380      C  Naypyitaw        Member    Male
989  430-53-4718      B   Mandalay        Member    Male
991  602-16-6955      B   Mandalay        Normal  Female
996  303-96-2227      B   Mandalay        Normal  Female
999  849-09-3807      A     Yangon        Member  Female

               Product line  Unit price  Quantity   Tax 5%      Total
\
0         Health and beauty       74.69         7  26.1415   548.9715
4         Sports and travel       86.31         7  30.2085   634.3785
5    Electronic accessories       85.39         7  29.8865   627.6165
7        Home and lifestyle       73.56        10  36.7800   772.3800
```

In [ ]:

In [ ]: